Software Design Principles

SOLID Principles

SOLID is a set of five design principles that guide developers in creating software that is modular, maintainable, and scalable. These principles are especially relevant in modern architectures like microservices, where independent services must remain robust and adaptable.

Single Responsibility Principle (SRP)

- **Definition**: A class or service should have only one reason to change. It must focus on a single business capability.
- **Example**:
    - User Service handles user registration and authentication only.
    - Order Service manages order placement and tracking.
- **Benefit**: Reduces complexity, improves testability, and makes debugging easier.
- **Visual Representation**: https://miro.medium.com/v2/resize:fit:800/1*YwYwYwYwYwYwYw.png

---

Open/Closed Principle (OCP)

- **Definition**: Software entities should be open for extension but closed for modification.
- **Application**: Instead of altering existing code, new functionality should be added via inheritance, interfaces, or plugins.
- **Example**:
    - Payment Service supports credit cards. To add PayPal, extend functionality without modifying the existing logic.
- **Benefit**: Prevents regression bugs and supports evolving business needs.

Liskov Substitution Principle (LSP)

- **Definition**: Objects of a superclass should be replaceable with objects of a subclass without breaking the system.
- **Example**:
    - A Bird class has a fly() method. A subclass Sparrow can substitute it. But a subclass Penguin should not inherit fly() since it violates expectations.
- **Benefit**: Ensures reliability and consistency in polymorphic behavior.

Interface Segregation Principle (ISP)

- **Definition**: Clients should not be forced to depend on interfaces they do not use.
- **Example**:
    - Instead of one large ServiceInterface with methods for logging, payments, and notifications, create smaller interfaces like PaymentInterface, NotificationInterface.
- **Benefit**: Reduces unnecessary dependencies and promotes modular design.

Dependency Inversion Principle (DIP)

- **Definition**: High-level modules should not depend on low-level modules. Both should depend on abstractions.
- **Example**:
    - An Order Service should depend on a PaymentProcessor interface, not directly on a StripePayment implementation.
- **Benefit**: Increases flexibility, supports dependency injection, and simplifies testing.

DRY (Don't Repeat Yourself) and KISS (Keep It Simple, Stupid) Principles

DRY Principle

- **Definition**: Avoid duplication of logic or code. Every piece of knowledge should have a single representation in the system.
- **Example**:
    - Instead of writing validation logic in multiple services, create a shared validation library.
- **Benefit**: Reduces redundancy, improves maintainability, and minimizes errors.

KISS Principle

- **Definition**: Keep designs simple and avoid unnecessary complexity.
- **Example**:
    - Use straightforward REST APIs instead of over-engineered communication protocols when not required.
- **Benefit**: Enhances readability, reduces bugs, and accelerates development.

Practical Application of DRY and KISS

- **Scenario**:

    - In microservices, DRY ensures shared utilities (like logging or authentication) are centralized.
    - KISS ensures each service remains lightweight and focused on its core responsibility.
- **Outcome**: Together, they balance efficiency and simplicity, preventing bloated architectures.

Designing for Scalability and Maintainability

Scalability in Design

- **Definition**: Ability of the system to handle growth in users, data, or workload.
- **Techniques**:
    - Horizontal scaling: Add more instances of services.
    - Asynchronous communication: Use message queues to decouple services.
    - Caching: Reduce repeated computations.
- **Example**:
    - E-commerce platforms scale order and payment services independently.
- **Image**: https://upload.wikimedia.org/wikipedia/commons/5/5e/Horizontal_vs_Vertical_Scaling.png

---

Maintainability in Design

- **Definition**: Ease with which software can be updated, debugged, and extended.
- **Techniques**:
    - Modular design: Independent services with clear boundaries.
    - CI/CD pipelines: Automated testing and deployment.
    - Observability: Centralized logging and tracing.
- **Example**:
    - A Notification Service can be updated without affecting Order or Payment services.

Patterns and Techniques

- **Service Registry**: Helps services discover each other dynamically.
- **Saga Pattern**: Manages distributed transactions across microservices.
- **Event-Driven Architecture**: Promotes loose coupling and real-time processing.
- **Circuit Breaker Pattern**: Prevents cascading failures by isolating faulty services.

- **API Gateway**: Provides a single entry point for clients, handling routing, authentication, and rate limiting.